



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

ROBÓTICA I

PROFESSOR: JURACY EMANUEL MAGALHÃES DA FRANCA

ALINE DE BRITO SÉRIO

**Desenvolvimento e Integração de uma Arquitetura Híbrida
de Navegação Autônoma utilizando A* Aprimorado e DWA
em Ambiente Simulado**

JUAZEIRO - BA

2026

1. Introdução

O desenvolvimento de sistemas de navegação para robôs móveis autônomos envolve dois desafios principais: encontrar uma rota global segura até o objetivo e, ao mesmo tempo, permitir que o robô consiga reagir localmente aos obstáculos presentes no ambiente.

Quando se utiliza apenas um planejador global, como o A*, o robô consegue encontrar um caminho completo do ponto inicial até o alvo, mas esse caminho não considera diretamente as restrições reais de movimento do robô, como velocidade, aceleração e raio de giro. Por outro lado, quando se utiliza apenas um planejador local, como o DWA, o robô consegue reagir aos obstáculos próximos, mas pode apresentar dificuldades para escolher uma direção global adequada, principalmente em ambientes com obstáculos que criam desvios mais complexos.

Dessa forma, este projeto propõe a integração de uma arquitetura híbrida de navegação, combinando o algoritmo A* Aprimorado com o algoritmo DWA Aprimorado. O A* foi utilizado como planejador global, responsável por gerar uma rota inicial no mapa de ocupação. Já o DWA foi utilizado como planejador local, responsável por transformar essa rota em comandos de velocidade linear e angular para o robô no ambiente simulado.

O sistema foi implementado em Python e integrado ao CoppeliaSim, permitindo validar o comportamento do robô em um cenário simulado com obstáculos. O objetivo principal foi demonstrar, na prática, como a combinação entre planejamento global e controle local melhora a navegação autônoma, tornando o deslocamento mais seguro, estável e eficiente.

2. Fundamentação Teórica

A navegação autônoma de robôs móveis pode ser dividida em duas etapas principais: planejamento de caminho e controle de movimento.

O planejamento de caminho é responsável por encontrar uma rota entre a posição inicial e o alvo, considerando a presença de obstáculos no ambiente.

Já o controle de movimento é responsável por fazer o robô seguir a rota planejada, respeitando suas limitações físicas e desviando de obstáculos próximos. Nessa etapa, é necessário considerar as velocidades possíveis do robô e prever se determinado comando pode causar colisão.

Neste projeto, essas duas etapas foram combinadas da seguinte forma:

- o A* Aprimorado atua como planejador global;
- o DWA Aprimorado atua como planejador local;
- o CoppeliaSim fornece o ambiente simulado;
- o Python executa os algoritmos e envia os comandos de controle ao robô.

Essa integração permite que o robô tenha uma direção global definida pelo A* e, ao mesmo tempo, consiga ajustar seu movimento localmente por meio do DWA.

3. Arquitetura Geral do Sistema

A arquitetura desenvolvida foi organizada em etapas, seguindo um fluxo lógico de execução. O sistema inicia com a comunicação entre o código Python e o CoppeliaSim. Em seguida, o ambiente é convertido em um mapa de ocupação, que representa as áreas livres e os obstáculos. Depois disso, o algoritmo A* calcula a rota global. Por fim, o DWA utiliza essa rota como referência para gerar os comandos de movimento do robô.

O fluxo geral do sistema pode ser representado da seguinte forma:

```
iniciar_conexao_com_coppeliasim()

mapa_ocupacao = gerar_mapa_do_ambiente()

rota_global = calcular_rota_astar(
    inicio,
    objetivo,
    mapa_ocupacao
)

while simulacao_ativa:

    pose_atual = obter_pose_do_robo()
```

```

if robo_chegou_ao_objetivo(pose_atual, objetivo):

    parar_robo()
    break

    waypoint_atual = selecionar_proximo_waypoint(

        rota_global,
        pose_atual

    )

    v, w = calcular_comando_dwa(

        pose_atual,
        waypoint_atual,
        mapa_ocupacao

    )

enviar_velocidades_para_o_robo(v, w)

```

Essa estrutura evidencia a divisão entre a etapa de planejamento global e a etapa de controle local. O A* não controla diretamente os motores do robô. Ele apenas fornece a rota de referência. O DWA, por sua vez, interpreta essa rota e escolhe as velocidades mais adequadas em cada instante da simulação.

4. Mapeamento do Ambiente

Para que o robô consiga planejar uma rota, o ambiente precisa ser transformado em uma representação computacional. Neste projeto, essa representação no cenário do CoppeliaSim que foi transformado em uma matriz, indicando quais regiões estão livres e quais regiões possuem obstáculos, para que o algoritmo A* pudesse calcular a rota do robô

A etapa de mapeamento foi essencial porque o algoritmo A* não trabalha diretamente sobre a cena tridimensional do CoppeliaSim. Ele precisa de uma

representação discreta do ambiente. Por isso, as informações do cenário foram processadas e convertidas em uma matriz utilizando Python e NumPy.

O resultado desse processo pode ser observado na imagem `captura_mapa_ocupacao.png`, que mostra a conversão do ambiente simulado em um mapa de ocupação. Essa imagem permite verificar se os obstáculos foram corretamente identificados e se as regiões livres foram preservadas para o planejamento da rota.

Essa etapa também foi importante para evitar que o caminho calculado passasse muito próximo das bordas dos obstáculos. Para isso, foi aplicada uma margem de segurança na região ocupada, aumentando a distância mínima entre o robô e os obstáculos durante a navegação.

5. Planejador Global: A* Aprimorado

O algoritmo A* foi utilizado para calcular a rota global entre o ponto inicial e o alvo. Ele funciona avaliando os possíveis caminhos em um mapa discreto e escolhendo os nós com menor custo estimado.

A função principal de avaliação do A* é dada por:

$$f(n) = g(n) + h(n)$$

Onde:

f(n) representa o custo total estimado.

g(n) representa o custo acumulado desde o início.

h(n) representa a estimativa de distância até o objetivo.

representa a estimativa de custo do nó atual até o objetivo.

Neste projeto, a heurística utilizada foi baseada na distância euclidiana entre o nó atual e o objetivo:

$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

Essa heurística permite estimar a distância direta até o alvo, orientando a busca para regiões mais promissoras do mapa.

Além disso, o A* foi aprimorado com uma penalização relacionada à proximidade dos obstáculos. Essa melhoria teve como objetivo evitar que a rota global passasse muito próxima das regiões ocupadas. Assim, o custo total passou a considerar não apenas a distância até o objetivo, mas também a segurança do trajeto.

De forma simplificada, o custo utilizado pode ser representado por:

$$f(n) = g(n) + h(n) + P(n)$$

Onde:

$[P(n)]$

representa uma penalidade associada à proximidade de obstáculos.

Com isso, o planejador global passou a favorecer caminhos mais seguros, mesmo que eles fossem um pouco mais longos. Essa escolha é importante em robótica móvel, pois uma rota matematicamente curta nem sempre é a mais adequada quando se considera o tamanho físico do robô e o risco de colisão.

O resultado do planejamento global pode ser observado na imagem `captura_mapa_com_rota.png`, na qual é possível visualizar a rota calculada pelo A* sobre o mapa de ocupação.

6. Planejador Local: DWA Aprimorado

Após o cálculo da rota global, foi necessário fazer o robô seguir esse caminho dentro do CoppeliaSim. Para isso, foi utilizado o algoritmo DWA, Dynamic Window Approach.

O DWA é um planejador local que trabalha avaliando possíveis velocidades para o robô. Em vez de escolher diretamente uma posição no mapa, ele escolhe comandos de velocidade linear e angular:

$$[(v, \omega)]$$

Onde:

$$[v]$$

É a velocidade linear do robô.

$$[\omega]$$

É a velocidade angular do robô.

A principal vantagem do DWA é que ele considera as limitações cinemáticas do robô. Ou seja, ele não escolhe movimentos impossíveis. As velocidades avaliadas ficam dentro de uma janela dinâmica, limitada pela velocidade atual, pela aceleração máxima e pelo intervalo de tempo da simulação.

A janela dinâmica pode ser representada por:

$$[V_d = \left\{ (v, \omega) \mid v \in [v_a - \dot{v}_{\max} \Delta t, v_a + \dot{v}_{\max} \Delta t] \text{ and } \omega \in [\omega_a - \dot{\omega}_{\max} \Delta t, \omega_a + \dot{\omega}_{\max} \Delta t] \right\}]$$

Onde:

$$[v_a]$$

é a velocidade linear atual.

$$[\omega_a]$$

é a velocidade angular atual.

$$[\dot{v}_{\max}]$$

é a aceleração linear máxima permitida.

$$[\dot{\omega}_{\max}]$$

é a aceleração angular máxima permitida.

$$[\Delta t]$$

é o intervalo de tempo utilizado para a previsão do movimento.

Para cada par de velocidades possível, o DWA simula uma pequena trajetória futura e avalia se ela é segura. Caso a trajetória prevista leve o robô a uma colisão, aquele par de velocidades é descartado.

A função de avaliação utilizada no DWA considera três critérios principais:

$$[G(v,\omega) = \alpha \cdot \text{heading}(v,\omega) + \beta \cdot \text{dist}(v,\omega) + \gamma \cdot \text{velocity}(v,\omega)]$$

Onde:

$$[\text{heading}(v,\omega)]$$

mede o alinhamento do robô em relação ao próximo waypoint da rota global.

$$[\text{dist}(v,\omega)]$$

mede a distância entre a trajetória prevista e o obstáculo mais próximo.

$$[\text{velocity}(v,\omega)]$$

favorece velocidades lineares maiores, evitando movimentos muito lentos quando não há risco de colisão.

$$[\alpha, \beta, \gamma]$$

são pesos utilizados para ajustar a importância de cada critério.

A principal melhoria aplicada no DWA foi utilizar os waypoints gerados pelo A* como subalvos locais. Dessa forma, o DWA não tenta seguir diretamente o objetivo final o tempo todo. Ele segue pequenos pontos intermediários, o que reduz oscilações e melhora a capacidade do robô de contornar obstáculos.

Essa integração é o ponto central do projeto: o A* fornece a estratégia global e o DWA executa a movimentação local de forma segura.

7. Integração com o CoppeliaSim

A validação prática do projeto foi realizada no ambiente CoppeliaSim. O robô simulado foi controlado por meio de um script em Python, responsável por enviar comandos de velocidade e receber informações da simulação.

Durante a execução, o código realiza as seguintes etapas:

1. conecta o Python ao CoppeliaSim;
2. identifica o robô e os elementos principais da cena;
3. gera o mapa de ocupação do ambiente;
4. calcula a rota global com o A*;
5. acompanha a pose atual do robô;
6. seleciona o próximo waypoint da rota;
7. calcula as velocidades com o DWA;
8. envia os comandos aos motores;
9. verifica se o robô chegou ao alvo.

Essa integração permitiu observar o comportamento do robô em tempo real, verificando se os comandos gerados pelo DWA eram suficientes para seguir a rota global calculada pelo A*.

8. Metodologia de Desenvolvimento

O desenvolvimento do projeto foi realizado de forma incremental. Inicialmente, o sistema foi testado em condições mais simples, com o ambiente livre de obstáculos. Essa etapa teve como objetivo verificar se o robô conseguia se movimentar e responder corretamente aos comandos de velocidade.

Após essa validação inicial, foram adicionados obstáculos de forma gradual. Essa estratégia permitiu identificar problemas de controle, como oscilações, paradas inesperadas e trajetórias muito próximas das regiões ocupadas.

Em seguida, foi feita a integração entre o planejador global e o planejador local. O A* passou a gerar a rota de referência e o DWA passou a utilizar os waypoints dessa rota para guiar o robô.

Essa abordagem incremental foi importante porque permitiu testar cada parte do sistema separadamente antes da integração completa. Assim, os erros puderam ser localizados com mais facilidade.

A metodologia adotada pode ser resumida nas seguintes etapas:

Etapas	Objetivo	Resultado esperado
Teste em ambiente livre	Verificar movimentação básica do robô	Robô se desloca sem obstáculos
Inserção de obstáculos simples	Avaliar desvio local	Robô evita colisão
Geração do mapa de ocupação	Representar o ambiente em grid	Obstáculos identificados no mapa
Planejamento com A*	Calcular rota global	Caminho livre até o alvo
Controle com DWA	Gerar velocidades seguras	Robô segue a rota
Integração A* + DWA	Validar navegação híbrida	Robô chega ao alvo sem colisão

9. Resultados Obtidos

Os resultados obtidos demonstraram que a integração entre o A* Aprimorado e o DWA Aprimorado melhorou o comportamento do robô em relação a uma abordagem puramente local.

Nos testes iniciais, quando o robô dependia apenas do comportamento reativo local, foram observadas oscilações e dificuldade para escolher uma direção adequada em alguns momentos. Isso ocorreu porque o planejador local analisava apenas uma região limitada do ambiente.

Com a inclusão da rota global gerada pelo A*, o robô passou a ter uma referência mais clara de deslocamento. O DWA deixou de tentar alcançar diretamente o objetivo final e passou a seguir waypoints intermediários. Isso tornou o movimento mais organizado e reduziu desvios desnecessários.

Os principais resultados observados foram:

Critério avaliado

Resultado observado

Geração do mapa de ocupação

O ambiente foi convertido em uma matriz discreta utilizável pelo A*

Planejamento global

O A* encontrou uma rota livre de obstáculos

Seguimento da rota

O DWA conseguiu utilizar os waypoints como referência

Desvio de obstáculos

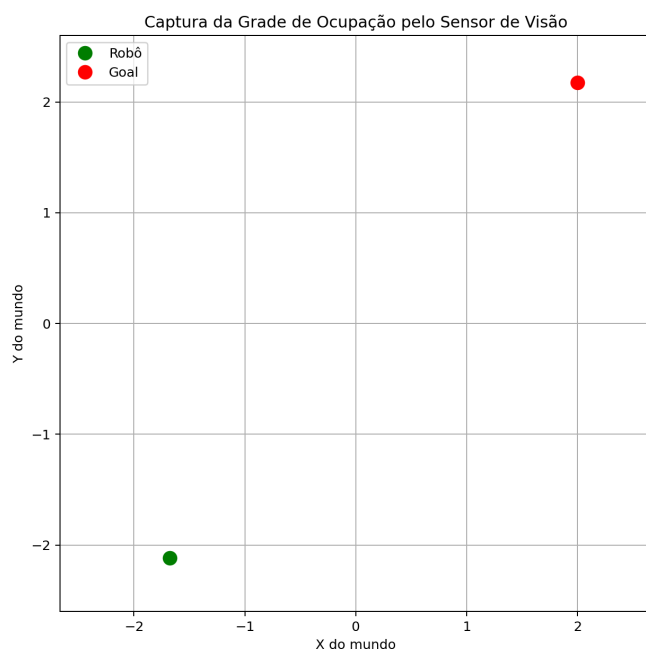
O robô realizou desvios sem colisão no cenário testado

Estabilidade do movimento

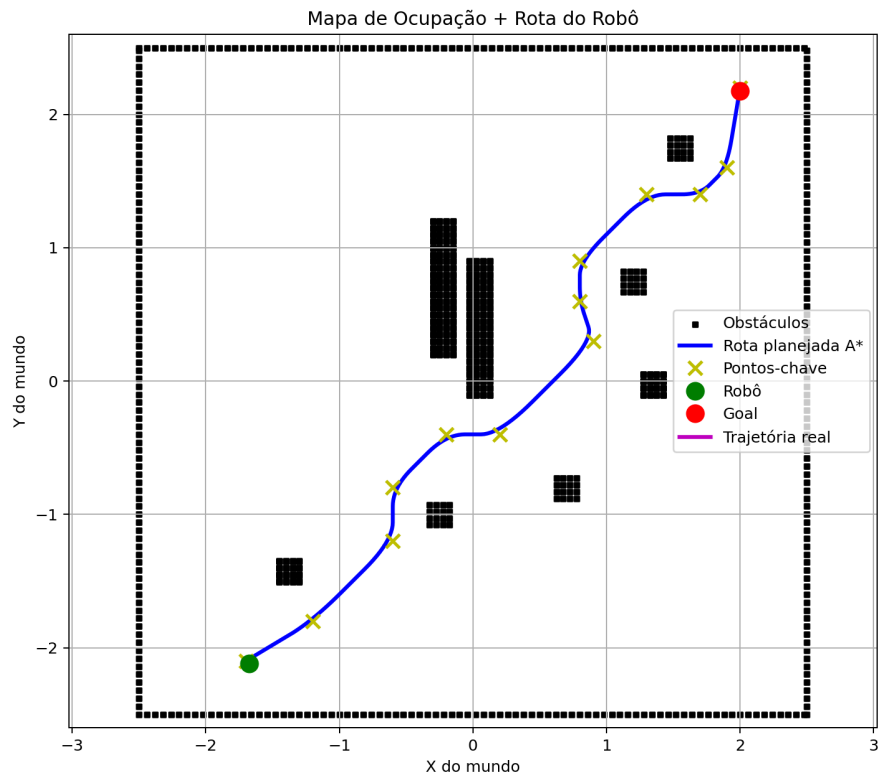
Houve redução de oscilações após a integração com o A*

Chegada ao alvo

O robô conseguiu convergir para a região do objetivo



A imagem apresenta a posição inicial do robô e a posição do objetivo no sistema de coordenadas do ambiente simulado. Essa etapa antecede o planejamento da rota, pois define os pontos de início e destino utilizados pelo algoritmo A*



A imagem `captura_mapa_com_rota.png` apresenta o resultado do planejamento global realizado pelo A*. Os obstáculos aparecem representados em preto, o ponto inicial do robô em verde, o objetivo em vermelho e a rota planejada em azul. Os pontos intermediários indicam os waypoints utilizados posteriormente pelo DWA como referências locais. Observa-se que a rota não atravessa regiões ocupadas e mantém desvio em relação aos obstáculos, evidenciando a atuação do planejador global aprimorado.

10. Relação entre o Código e os Algoritmos

A implementação em Python permitiu transformar os conceitos teóricos em um sistema funcional. No código, o A* aparece principalmente na etapa de busca sobre o grid de ocupação. A cada nó analisado, o algoritmo calcula o custo acumulado, a heurística e a penalização relacionada aos obstáculos.

De forma simplificada, essa lógica pode ser representada por:

$$f = g + h + \text{penalidade}$$

Esse valor é utilizado para decidir qual nó será explorado primeiro. O uso de uma fila de prioridade permite que o algoritmo sempre selecione o nó com menor custo estimado.

Já o DWA aparece na etapa de escolha das velocidades do robô. O algoritmo testa diferentes combinações de velocidade linear e angular, prevê a trajetória resultante e calcula um custo para cada possibilidade.

A lógica principal pode ser representada por:

```
for v in velocidades_lineares:
    for w in velocidades_angulares:
        trajetoria = prever_movimento(pose_atual, v, w)
        if existe_colisao(trajetoria, mapa):
            custo = infinito
        else:
            custo = calcular_custo_total(trajetoria, waypoint, obstaculos)
```

A melhor velocidade é aquela que apresenta o menor risco de colisão e melhor alinhamento com o próximo waypoint.

Assim, o código evidencia a aplicação prática dos dois algoritmos estudados. O A* decide por onde o robô deve passar. O DWA decide como o robô deve se mover para seguir esse caminho.

11. Discussão

A arquitetura híbrida apresentou uma vantagem importante em relação ao uso isolado dos algoritmos. O A* sozinho gera uma rota eficiente, mas não controla diretamente o movimento do robô. Já o DWA sozinho consegue evitar obstáculos próximos, mas pode apresentar dificuldade em ambientes mais complexos, pois não possui uma visão global da rota.

Ao integrar os dois métodos, o sistema passou a combinar as vantagens de ambos. O A* forneceu a orientação global, enquanto o DWA garantiu a execução local considerando as restrições de movimento do robô.

Outro ponto importante foi a utilização do mapa de ocupação. Essa representação tornou possível aplicar o A* sobre o ambiente simulado e permitiu visualizar de forma clara a relação entre obstáculos, áreas livres e rota planejada.

Apesar dos bons resultados no ambiente simulado, o sistema ainda possui limitações. A validação foi realizada em um cenário controlado, dentro do CoppeliaSim. Para aplicações reais, seria necessário considerar ruídos de sensores, imprecisão na odometria, obstáculos dinâmicos e diferenças entre o modelo simulado e o comportamento físico de um robô real.

Mesmo assim, dentro do escopo proposto para a disciplina, o projeto demonstrou corretamente a integração entre planejamento global, controle local e simulação robótica.

12. Conclusão

O projeto desenvolvido cumpriu o objetivo de implementar e validar uma arquitetura híbrida de navegação autônoma utilizando A* Aprimorado e DWA Aprimorado em ambiente simulado.

O A* foi responsável por calcular uma rota global segura a partir do mapa de ocupação, enquanto o DWA foi responsável por gerar os comandos de velocidade linear e angular para o robô seguir essa rota no CoppeliaSim. A utilização dos waypoints do A* como referência para o DWA melhorou o comportamento do robô, reduzindo oscilações e tornando o deslocamento mais organizado.

A etapa de mapeamento também foi fundamental, pois permitiu transformar o ambiente simulado em uma matriz discreta utilizada pelo planejador global. As imagens geradas durante o projeto comprovam as principais etapas do sistema: o mapa de ocupação, a rota planejada e a execução do robô no simulador.

Portanto, a integração entre A* e DWA mostrou-se adequada para o problema de navegação proposto. O sistema desenvolvido demonstra, de forma prática, a

importância de combinar planejamento global e controle local em robôs móveis autônomos.

Como melhorias futuras, poderiam ser adicionados testes quantitativos mais detalhados, análise de tempo de execução, comparação entre DWA puro e A^* + DWA, além da inclusão de obstáculos dinâmicos para aproximar o sistema de situações reais de navegação.

13. Considerações Finais

A principal contribuição do projeto foi demonstrar, na prática, que a navegação autônoma se torna mais eficiente quando o robô utiliza uma estratégia híbrida. O planejador global define a rota geral, enquanto o planejador local executa o movimento de forma segura e compatível com as limitações do robô.

Dessa forma, o projeto não se limita a uma demonstração visual no simulador, mas representa a aplicação direta de conceitos fundamentais da robótica móvel: mapeamento, planejamento de caminho, controle cinemático, desvio de obstáculos e integração entre software e ambiente de simulação.

14. Referências

BAI, X.; JIANG, H.; CUI, J.; LU, K.; CHEN, P.; ZHANG, M. **UAV Path Planning Based on Improved A^* and DWA Algorithms**. *International Journal of Aerospace Engineering*, v. 2021, Article ID 4511252, 2021.

FOX, D.; BURGARD, W.; THRUN, S. **The Dynamic Window Approach to Collision Avoidance**. *IEEE Robotics & Automation Magazine*, v. 4, n. 1, p. 23–33, 1997.

SAKAI, A. et al. **PythonRobotics: a Python code collection of robotics algorithms**. 2018.

COPPELIASIM. **CoppeliaSim User Manual**. Documentação oficial do simulador.

PYTHON SOFTWARE FOUNDATION. **Python Documentation**. Documentação oficial da linguagem Python.

NUMPY DEVELOPERS. **NumPy Documentation**. Documentação oficial da biblioteca NumPy.